.NET 10: from first steps to a complete project

Original learning guide | 14 October 2026 | Read, run, change, explain, practice

All commands assume a PowerShell terminal at the repository root. Code is included for study; use the source files for copying and editing.

10 lessons, ordered from easy to hard. Read DotNet10-Learning-Guide.pdf for the complete printable course, or use the individual lesson files below.

Use this track after C# lessons 01 through 08 and PowerShell lessons 01 through 06. .NET is a platform rather than a separate programming language. This track teaches projects, libraries, configuration, dependency injection, APIs, clients, testing, and publishing. The projects target net10.0 and use installed SDK/shared-framework components without third-party packages.

Before lesson 01

1. Open a PowerShell terminal at the repository root, the folder containing LearningPack.slnx.
2. If local scripts are blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` for this terminal. This changes only this process, not the machine policy.
3. Run `. .\Initialize-Training.ps1` (there is a space between the first and second dots).
4. Keep the terminal at the repository root when copying commands from lessons.
5. For every lesson: predict the output, run it, read each statement, change one thing, complete the exercise, then check SOLUTION.md.

Lesson order

1. 01-Sdk: The SDK, runtime, and command line
2. 02-Library: A library and project references
3. 03-Configuration: JSON settings and environment variables
4. 04-DependencyInjection: Dependency injection and logging
5. 05-MinimalApi: Your first HTTP API
6. 06-Validation: Validate request bodies and status codes
7. 07-HttpClient: Call an API from C#
8. 08-Testing: Executable checks and failure cases
9. 09-Publishing: Release builds and framework-dependent publishing
10. 10-Capstone: A persistent task API

How to know you are ready to move on

Explain the example without reading the notes. Complete the exercise without copying the solution. Make one valid change and one invalid-input experiment. Record what happened in Progress.md. If you cannot explain a result, repeat the lesson with smaller values.

Learn Git with this track

Read the Git workflow and this track's Git checkpoints. Each lesson includes a branch name, verification step, staging paths, and suggested commit. The printable course guide includes the shared Git workflow too.

Official PDF references

The Official-References index lists downloaded publisher PDFs and printable copies of current Microsoft documentation, with source URLs, editions, and course mappings. Publisher PDFs retain their original content. Documentation exports are clearly labeled as local print copies, rather than Microsoft-published PDF books.

Reference documentation

The lesson guides are original teaching material. Official downloads and documentation exports are listed in Official-References/README.md. The links below provide further online study. Checked on 14 October 2026.

- .NET 10 overview
- Install .NET on Windows
- .NET CLI
- File-based applications
- ASP.NET Core minimal APIs
- Dependency injection
- Publishing

Course code style

Authored C# follows Google's C# guide, linked from the Google style-guide index.

C# and .NET lessons

- Use two spaces, same-line opening braces, and a 100-column limit.
- Put one statement on each line and brace control-flow bodies.
- Use PascalCase for types/public members, camelCase for parameters/locals, and _camelCase for private fields.
- Place System imports first and sort other imports alphabetically.
- Use block-bodied methods; reserve expression bodies for suitable properties and simple lambdas.
- Put each core type in its own matching source file.
- Use constants for fixed rules and readonly fields for stable dependencies.

.editorconfig records formatter settings and private-field naming. Types now live in files such as StudySession.cs, StudyEntry.cs, and TaskRules.cs. Numbered folders preserve lesson order.

The course still teaches modern C#: top-level programs, file-scoped namespaces, records, collection expressions, and C# 14 field-backed properties. Positional record parameters use PascalCase because they declare public properties.

PowerShell convention

Google's shell guide targets Bash, so these scripts use a documented course convention: two spaces, same-line braces, explicit blocks, and separate statements. Aim for 100 columns; long literal URLs, generated HTML here-strings, and indivisible arguments can exceed that target.

Keep Verb-Noun function names, PascalCase parameters/output properties, and camelCase locals. Use full commands, structured output, help comments, validation, native exit-code checks, and splatting for long calls.

Verify before committing

```
. .\Initialize-Training.ps1
dotnet build .\LearningPack.slnx -m:1 -nr:false
.\Check-Style.ps1
.\Test-LearningPack.ps1
```

To apply the SDK formatter:

```
dotnet format whitespace .\LearningPack.slnx --no-restore
```

```
dotnet format style .\LearningPack.slnx --no-restore --diagnostics IDE0011 IDE0022 IDE0061 --severity
info
```

Review changes and run checks before regenerating course PDFs with Build-Guides.ps1. Original publisher PDFs and official documentation exports retain their source formatting.

Git practice in the DotNet10 course

Read Git-Workflow.md first and save the supplied baseline before editing. Start each exercise on a clean branch from main. Run focused checks, stage related changes, commit with a clear message, then choose either local integration or a GitHub pull request.

Track checkpoints

Lesson	Branch	Suggested commit	Extra Git skill
01-Sdk	feat/dotnet10-01-sdk-notes	docs(dotnet10): explain project SDK settings	Inspect status, diff, and staging
02-Library	feat/dotnet10-02-remaining-minutes	feat(dotnet10): add a remaining minutes calculation	Inspect status, diff, and staging
03-Configuration	feat/dotnet10-03-environment-override	test(dotnet10): verify the daily target override	Inspect status, diff, and staging
04-DependencyInjection	feat/dotnet10-04-service-lifetimes	test(dotnet10): compare service lifetimes	Keep commits focused and review the staged snapshot
05-MinimalApi	feat/dotnet10-05-minutes-route	feat(dotnet10): add a minutes conversion endpoint	Keep commits focused and review the staged snapshot
06-Validation	feat/dotnet10-06-invalid-input	test(dotnet10): verify invalid session requests	Keep commits focused and review the staged snapshot
07-HttpClient	feat/dotnet10-07-connection-failure	test(dotnet10): document client failure behavior	Push a branch and explain validation in a PR
08-Testing	feat/dotnet10-08-boundary-cases	test(dotnet10): check study budget boundaries	Push a branch and explain validation in a PR
09-Publishing	feat/dotnet10-09-publish-notes	docs(dotnet10): explain published runtime files	Push a branch and explain validation in a PR
10-Capstone	feat/dotnet10-10-task-summary	feat(dotnet10): add task summary counts	Review integration history and practice conflict resolution

For a docs-only change, prefer a docs/ branch; for a bug fix, prefer fix/. The suggested feat/ name is a convenient starting point and can be adapted to what you actually implement. Do not describe a documentation-only experiment as a new program feature.

Practice in a separate local repository

```
.\New-GitPractice.ps1 -Topic DotNet10
```

Use the printed developer and teammate paths. Both clones and their bare remote are inside .artifacts. Follow the push, merge, and conflict examples from Git-Workflow.md there. This is separate from changing your actual course repository.

A complete checkpoint

1. Verify the change with the lesson commands and an appropriate boundary case.
2. Inspect git diff and stage only related source or notes.
3. Inspect git diff --cached and commit.
4. Push your branch if you intend to publish it.
5. Review a PR or perform a local merge, then verify the behavior on main.
6. Record the resulting commit ID in Progress.md.

Do not commit generated build output, demonstration data, API logs, browser profiles, or credentials. Regenerate the original course PDFs after documentation changes only when the source content is ready; inspect PDF changes by size and rendered pages rather than a text diff.

Git workflow for all three courses

Use Git while completing exercises. Git records versions locally; GitHub hosts a remote repository and adds pull requests. A branch lets you develop a change before integrating it into main. A commit stores the staged snapshot with a message. Push sends commits to a remote. Merge combines histories.

This course uses short-lived branches, small commits, and reviewed merges. It is one practical workflow; team rules can differ. The optional type(scope): summary message convention is a course recommendation, not a Git requirement.

1. Inspect and set your identity

```
git --version
git status --short --branch
git branch --show-current
git remote -v
git log --oneline --decorate -5
```

The working tree is your files on disk. The index, or staging area, holds the next proposed snapshot. HEAD identifies the current commit. origin is a conventional remote name. This repository already has origin: check its actual address instead of adding another remote. Do not run git init inside the lesson folders; all courses belong to one repository.

If identity is missing, replace these placeholders with your own name and verified or GitHub noreply email:

```
git config --local user.name "Your Real Name"
git config --local user.email "your-verified-or-GitHub-noreply-email"
git config --get user.name
git config --get user.email
```

Local identity settings affect only this repository. Commit identity differs from push authentication. Use Git Credential Manager or GitHub's supported authentication; keep passwords and tokens out of source files and remote URLs.

2. Commit the supplied baseline before exercises

An untracked file does not exist in Git history. If the learning pack is initially uncommitted, save its supplied state before editing lessons:

```
git status --short --branch
git switch -c course/learning-pack
git add -- .gitignore .gitattributes .editorconfig README.md STYLE.md Directory.Build.props global.json
NuGet.Config LearningPack.slnx
git add -- Initialize-Training.ps1 Test-LearningPack.ps1 Check-Style.ps1 Build-Guides.ps1
```

```
git add -- Git-Workflow.md New-GitPractice.ps1 Official-Resources.json Download-OfficialReferences.ps1
git add -- CSharp PowerShell DotNet10 .github
git diff --cached --stat
git diff --cached --check
git commit -m "chore(course): add CSharp PowerShell and .NET 10 learning pack"
```

Review every staged path, including deletions. Confirm bin, obj, output, .tools and .artifacts are excluded. Inspect PDF sizes. Add only paths that exist in your copy.

Choose ONE baseline integration path:

- Local practice: git switch main, then git merge --no-ff course/learning-pack -m "Merge initial learning pack".
- GitHub practice: git push -u origin course/learning-pack, open a pull request to main, merge on GitHub, then fetch and fast-forward your local main.

Do not merge locally and also squash the same change on GitHub: those are different histories. If the baseline is already committed on main, skip this section.

3. Start a focused branch

Start with a clean working tree. Commit your current work or deliberately stash it before switching.

```
git switch main
git status --short
# For GitHub practice; omit fetch/pull in local-only practice:
git fetch origin
git pull --ff-only origin main
git switch -c feat/csharp-01-city
```

--ff-only refuses to combine divergent local and remote histories silently. If it refuses, inspect the graph and follow the synchronization section. Names such as feat/csharp-01-city, fix/powershell-09-minute-validation, and feat/dotnet10-task-summary identify one purpose. Branches are repository-wide, not restricted to the named folder. Integrate a lesson branch before starting the next unless you intentionally need a dependent branch.

4. Edit, verify, stage, and commit

For the first C# exercise, implement the optional city argument and verify both explicit and default inputs:

```
. .\Initialize-Training.ps1
dotnet run --project .\CSharp\01-Hello -- Dejan Ljubljana
dotnet run --project .\CSharp\01-Hello -- Dejan
git diff -- CSharp/01-Hello/Program.cs
git add -- CSharp/01-Hello/Program.cs
git diff --cached -- CSharp/01-Hello/Program.cs
git diff --cached --check
git status --short
```

```
git commit -m "feat(csharp): accept an optional city argument" -m "Use Ljubljana when the city is
omitted. Verified explicit and default inputs."
git show --stat HEAD
```

diff shows unstaged edits. diff --cached shows exactly what a commit will contain. Untracked files need editor inspection and staging before they appear in the staged diff. If you edit after staging, add again for edits you want in the same snapshot. git add -p stages selected hunks from tracked files.

Keep each commit focused and working. Use the lesson's verification commands. The root test script checks the supplied behavior; deliberate exercise changes may require updating its expectations.

5. Write clear messages and PR comments

Use an imperative summary describing the resulting change. A roughly 50-72-character summary is a useful readability target, not a Git limit. A body explains why a non-obvious decision was made and what relevant checks passed. Each additional -m adds another paragraph.

Type	Purpose	Example
feat	New behavior	feat(csharp): accept an optional city argument
fix	Correct behavior	fix(powershell): reject negative study minutes
docs	Documentation	docs(dotnet10): explain API validation responses
refactor	Structure with preserved behavior	refactor(csharp): extract study calculations
test	Verification	test(dotnet10): check task persistence
chore	Maintenance	chore(course): normalize source line endings

This is an optional convention. Avoid vague messages such as updates, fix stuff, or final. Source comments explain implementation details; commit messages explain a recorded change; PR comments discuss a proposed change and review. They have different purposes.

For a PR, state the problem, resulting behavior, and exact validation. The supplied .github/PULL_REQUEST_TEMPLATE.md provides those fields. Example title: Accept an optional city in the greeting. Example description: The greeting accepts a second argument and defaults to Ljubljana. Verified explicit and omitted city runs.

6. Push and review a pull request

```
git remote -v
git push -u origin feat/csharp-01-city
```

-u records the upstream; subsequent git push uses that relationship. Push publishes commits but does not merge main. On GitHub, create a pull request with main as base and your feature branch as head. Review Files changed, fix failed checks, and respond to comments. Additional pushes to that branch update the PR.

If a push is rejected, read the error: authentication, branch protection, and remote changes require different fixes. Fetch and inspect before attempting any history rewrite.

7. Merge: choose one completion path

Method	History effect	Useful context
Fast-forward	Moves the target pointer to a descendant	Simple local exercises
Merge commit	Records two parent histories	Preserve individual commits and integration point
Squash merge	Creates one new combined commit	One compact reviewed feature
Rebase then merge	Replays commits on a new base	Teams requiring linear history

Use a merge commit in the local lab to see branch history. For a small GitHub PR, squash merge is a reasonable default if repository policy allows it. Review the final squash message because it becomes main's commit message.

Local completion, without publishing:

```
git switch main
git status --short
git merge --no-ff feat/csharp-01-city -m "Merge CSharp city argument exercise"
git log --oneline --graph --decorate -10
git branch -d feat/csharp-01-city
```

Merge changes your CURRENT branch. --no-ff requests an explicit merge commit. Delete a branch only after integration. -d checks ancestry. After a GitHub squash merge, -d may refuse because the original commits were not merged by identity; keeping the branch is fine.

GitHub completion: merge the PR on GitHub first, then:

```
git fetch origin
git switch main
git pull --ff-only origin main
git log --oneline --graph --decorate -10
git status --short
```

Check the completed behavior is on main before starting the next lesson. git fetch --prune removes stale remote-tracking names, not your local branches.

To incorporate updated main into an active feature branch:

```
git switch feat/csharp-01-city
git fetch origin
git merge origin/main
# Resolve conflicts if needed, verify, and finish the merge.
git push
```

Rebase is another choice for a private, never-pushed branch. It changes commit IDs. This course uses merge to update published branches without rewriting their shared history.

8. Resolve conflicts and undo deliberately

A conflict means overlapping changes need your decision. Run status, open the file, find <<<<<<<, =======, and >>>>>>>, write the intended combined result, remove the markers, and verify it. Stage each resolved file and commit:

```
git status
# Edit and run checks BEFORE staging:
git add -- CSharp/01-Hello/Program.cs
git diff --cached
git commit -m "merge(csharp): preserve city input and updated greeting"
```

git merge --abort abandons the merge where possible. Start merges with a clean working tree so the pre-merge state is clear. Practice your first conflict in the isolated lab.

Command	Effect	Appropriate use
git restore --staged -- path	Unstage while keeping working edits	Wrong path staged
git restore -- path	Discard unstaged tracked-file edits	Only after reviewing and choosing to discard
git stash push -u -m "lesson draft"	Park edits and untracked files locally	Switch context temporarily
git stash list	Show parked work	Find your draft
git stash apply	Restore the latest stash while keeping it	Resume, inspect, then drop after verification
git revert commit-id	Record the inverse of a normal commit	Undo shared work without rewriting history

Stash is local temporary storage, not a pushed backup. Reverting a merge requires additional parent selection; the simple revert example is for a normal commit. Amend rewrites the latest commit; reserve it for appropriate private, unpushed work. Routine beginner work does not require reset --hard, clean -fd, or force pushes.

9. Practice pushing and conflicts offline

New-GitPractice.ps1 creates a fresh developer repository, bare local remote, and teammate clone under .artifacts. It copies your selected topic's first runnable lesson and configures fictional identities only inside those lab repositories. It does not change the course repository's branch or contact GitHub.

```
.\New-GitPractice.ps1 -Topic CSharp
# Use the developer path printed by the script:
Set-Location 'PASTE-THE-PRINTED-DEVELOPER-FOLDER'
git switch -c feat/practice-note
Add-Content -LiteralPath PracticeNotes.txt -Value 'I can explain the staging area.'
git diff
git add -- PracticeNotes.txt
git diff --cached
git commit -m "docs(csharp): record staging practice"
git push -u origin feat/practice-note
```

```
git switch main
git merge --no-ff feat/practice-note -m "Merge staging practice"
git push origin main
git log --oneline --graph --all
```

Open another terminal in the printed teammate folder. Run git pull --ff-only origin main and inspect PracticeNotes.txt. This demonstrates the same data movement without publishing to your GitHub account.

In the lab developer folder, create a deliberate conflict:

```
git switch main
git switch -c feat/conflict-note
Set-Content -LiteralPath PracticeNotes.txt -Value 'Feature: validate inputs before saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe validation"
git switch main
Set-Content -LiteralPath PracticeNotes.txt -Value 'Main: verify persistence after saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe persistence"
git merge feat/conflict-note
# EXPECT a conflict and nonzero exit code. Resolve the file:
git status
Get-Content -LiteralPath PracticeNotes.txt
Set-Content -LiteralPath PracticeNotes.txt -Value 'Validate inputs before saving, then verify
persistence.'
git add -- PracticeNotes.txt
git commit -m "merge(course): combine validation and persistence notes"
git log --oneline --graph --all
```

Both branches replaced the same text differently. Your resolution preserves both ideas. Return to the original repository root when finished.

10. Apply Git in every course

Each lesson contains a Git checkpoint with a branch name, focused verification, staging paths, and suggested message. Git-Practice.md maps your track. Use lessons 01-03 for status, branches, and staging; middle lessons for focused commits and review; final lessons for PRs, integration, and conflict practice.

For a combined API/client change, one focused branch can include both DotNet10 and PowerShell files because they implement one contract. Include only related changes. Keep generated files and secrets out of history.

Official references

This is original instructional material. Further detail comes from Git's command manuals and GitHub's workflow guide:

- git status
- git add

- git commit
- git merge
- git push
- git fetch
- git rebase
- GitHub flow

DotNet10: official reference PDFs

This folder contains **10 PDFs** from official source material. Files labeled Publisher PDF are the original downloaded publications. Files labeled Documentation export are locally printed copies of official Microsoft Learn articles, with source links, attribution, retrieval dates, and print formatting. They are not Microsoft-published PDF books or complete product manuals.

Read in this order

Use the course's original learning guide and runnable lessons first. Then consult the relevant current Microsoft documentation article for a deeper explanation. Read standards and architecture books when you want formal language/runtime details or further architecture study.

ECMA-335's sixth edition is a 2012 runtime standard, not the .NET 10 library catalog. The Microsoft architecture ebook covers an earlier generation of ASP.NET Core; use the .NET 10 overview and ASP.NET Core 10 tutorial for current APIs.

PDF catalog

PDF	Kind	Read with course lessons	Scope / edition
ECMA-335-CLI-6th-Edition.pdf	Publisher PDF	01-02; advanced reference	Common Language Infrastructure standard, 6th edition (June 2012). Runtime concepts and library profiles; not the .NET 10 API reference.
Microsoft-ASPNetCore-Architecture-Ebook.pdf	Publisher PDF	After 10; architecture	Microsoft's published ASP.NET Core/Azure architecture ebook. Earlier-generation guidance; consult current ASP.NET Core 10 documentation for API details.
Microsoft-DotNet10-Overview.pdf	Documentation export	01 and 10	Microsoft Learn .NET 10 overview.
Microsoft-DotNet-CLI.pdf	Documentation export	01-02 and 09	Microsoft Learn .NET command-line interface.
Microsoft-SystemTextJson.pdf	Documentation export	03 and 10; CSharp 10-12	Microsoft Learn System.Text.Json overview.
Microsoft-Dependency-Injection.pdf	Documentation export	04	Microsoft Learn dependency injection overview.
Microsoft-Configuration.pdf	Documentation export	03-04	Microsoft Learn configuration in .NET.
Microsoft-Logging.pdf	Documentation export	04-10	Microsoft Learn logging in .NET.
Microsoft-HttpClient.pdf	Documentation export	07	Microsoft Learn HttpClient guidelines.

Microsoft-ASPNetCore10-MinimalAPI.pdf	Documentation export	05-06 and 10	Microsoft Learn minimal API tutorial explicitly pinned to ASP.NET Core 10.

Sources

- ECMA-335-CLI-6th-Edition.pdf: official source and PDF download.
- Microsoft-ASPNetCore-Architecture-Ebook.pdf: official source and PDF download.
- Microsoft-DotNet10-Overview.pdf: official source.
- Microsoft-DotNet-CLI.pdf: official source.
- Microsoft-SystemTextJson.pdf: official source.
- Microsoft-Dependency-Injection.pdf: official source.
- Microsoft-Configuration.pdf: official source.
- Microsoft-Logging.pdf: official source.
- Microsoft-HttpClient.pdf: official source.
- Microsoft-ASPNetCore10-MinimalAPI.pdf: official source.

Provenance and refresh

Download-Record.json records the source URL, retrieval timestamp in UTC, file size, PDF SHA-256 hash, and original HTML hash for each documentation export. The retrieval timestamp records when the copy was collected; it is not the document's publication date. Article metadata dates are preserved where supplied. The official article remains the source of truth for changes after this snapshot.

From the repository root:

```
.\Download-OfficialReferences.ps1
```

This refreshes the collection using Official-Resources.json. It requires internet access and local Chrome or Edge. It downloads only the public documents and creates local print copies; it does not install software or publish anything.

Attribution

Original publisher PDFs retain their own copyright, license notices, and source formatting. Ecma standards include their copying terms. Microsoft Learn article wording and examples are retained while navigation is removed and print styling and a provenance banner are added. Credit belongs to Microsoft and the documentation contributors, including named authors credited in the source. See the Microsoft Learn terms and the .NET documentation CC BY 4.0 license. This course and its exports do not imply publisher endorsement.

01-Sdk: The SDK, runtime, and command line

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn understanding the platform and project lifecycle. Complete the setup in the track README.

Understand the idea

C# is a language. .NET is a platform containing a runtime and libraries. The SDK includes tools to create, build, and publish applications. A runtime installation alone cannot compile your code. A .csproj is an XML project definition. TargetFramework net10.0 selects .NET 10; the SDK supplies the matching C# compiler. dotnet restore resolves dependencies, build compiles, and run starts an application. Build output is placed in bin; intermediate files appear in obj. This course disables external package sources because its projects only use the installed SDK and shared frameworks. .NET 10 also supports file-based C# apps, demonstrated in Hello.cs.

Step by step

Hello.cs uses .NET 10 file-based property directives to disable Native AOT and self-contained output. This keeps the example runnable with the installed SDK and runtime, without downloading additional compiler or runtime packs.

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet --info
dotnet build .\DotNet10\01-Sdk
dotnet run --project .\DotNet10\01-Sdk
dotnet run --file .\DotNet10\01-Sdk\Hello.cs
```

Expected result

```
The project prints .NET major version: 10 and the SDK/runtime explanation. The file-based app prints
Hello from a .NET 10 file-based app!
```

Exercise

Inspect the csproj and identify TargetFramework, Nullable, and ImplicitUsings. Create a separate scratch console project under this lesson's output folder.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Set DOTNET_CLI_HOME locally if the SDK cannot access its user cache.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-01-sdk-notes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-01-sdk-notes
# Implement the exercise, then verify:
dotnet run --project .\DotNet10\01-Sdk
git diff -- DotNet10/01-Sdk/README.md DotNet10/01-Sdk/Program.cs
git add -- DotNet10/01-Sdk/README.md DotNet10/01-Sdk/Program.cs
git diff --cached
git diff --cached --check
git commit -m "docs(dotnet10): explain project SDK settings"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-01-sdk-notes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?

- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.01.Sdk.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup><Compile Remove="Hello.cs" /><Compile Remove="output/**/*.cs" /></ItemGroup>
</Project>
```

Hello.cs

```
#:property PublishAot=false
#:property SelfContained=false
Console.WriteLine("Hello from a .NET 10 file-based app!");
```

Program.cs

```
Console.WriteLine($".NET major version: {Environment.Version.Major}");
Console.WriteLine("The SDK builds applications; the runtime executes them.");
```

01-Sdk exercise solution

Inspect the csproj and identify TargetFramework, Nullable, and ImplicitUsings. Create a separate scratch console project under this lesson's output folder.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
New-Item -ItemType Directory -Path .\DotNet10\01-Sdk\output -Force | Out-Null
$env:DOTNET_CLI_HOME = Join-Path (Get-Location).Path '.tools\dotnet-home'
dotnet new console --framework net10.0 --output .\DotNet10\01-Sdk\output\Scratch
dotnet run --project .\DotNet10\01-Sdk\output\Scratch
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

02-Library: A library and project references

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn reusable assemblies and solution organization. Complete 01-Sdk first.

Understand the idea

A class library compiles reusable code without an executable entry point. A console app can reference it through ProjectReference. The compiler checks the dependency and builds it in the correct order. A namespace groups related types and helps avoid naming conflicts. public exposes library types to consumers. A solution groups projects for editor navigation and one-command builds; it does not replace project references. The Library folder contains StudyBudget, while App contains the executable. Keep business rules in the library and terminal-specific presentation in the app. You can test a library without running a terminal interface.

Step by step

1. Open App/Program.cs and Library/StudyBudget.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\02-Library\App
```

Expected result

```
3 sessions x 25 minutes = 75 minutes
Goal reached: True
```

Exercise

Add RemainingMinutes(completed, goal), never returning a negative result. Reject negative inputs. Use it from App.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use Math.Max after validating both inputs.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-02-remaining-minutes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-02-remaining-minutes
# Implement the exercise, then verify:
dotnet run --project .\DotNet10\02-Library\App
git diff -- DotNet10/02-Library/README.md DotNet10/02-Library/Library/StudyBudget.cs DotNet10/02-Library/App/Program.cs
git add -- DotNet10/02-Library/README.md DotNet10/02-Library/Library/StudyBudget.cs DotNet10/02-Library/App/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(dotnet10): add a remaining minutes calculation"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-02-remaining-minutes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

App/DotNet10.02.App.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup><ProjectReference Include="../Library/StudyMath.csproj" /></ItemGroup>
</Project>
```

App/Program.cs

```
using StudyMath;
Console.WriteLine($"3 sessions x 25 minutes = {StudyBudget.TotalMinutes(3, 25)} minutes");
Console.WriteLine($"Goal reached: {StudyBudget.HasReachedGoal(75, 60)}");
```

Library/StudyBudget.cs

```
namespace StudyMath;

public static class StudyBudget {
  public static int TotalMinutes(int sessions, int minutesPerSession) {
    if (sessions < 0) {
      throw new ArgumentOutOfRangeException(nameof(sessions));
    }

    if (minutesPerSession < 0) {
      throw new ArgumentOutOfRangeException(nameof(minutesPerSession));
    }

    return checked(sessions * minutesPerSession);
  }
  public static bool HasReachedGoal(int completed, int target) {
    if (completed < 0) {
      throw new ArgumentOutOfRangeException(nameof(completed));
    }

    if (target < 0) {
      throw new ArgumentOutOfRangeException(nameof(target));
```

```
    }

    return completed >= target;
  }
}
```

Library/StudyMath.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

02-Library exercise solution

Add RemainingMinutes(completed, goal), never returning a negative result. Reject negative inputs. Use it from App.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
public static int RemainingMinutes(int completed, int goal) {
  if (completed < 0) {
    throw new ArgumentOutOfRangeException(nameof(completed));
  }

  if (goal < 0) {
    throw new ArgumentOutOfRangeException(nameof(goal));
  }

  return Math.Max(0, goal - completed);
}
// Add inside StudyBudget; call it from App's Program.cs.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

03-Configuration: JSON settings and environment variables

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn configuration as data and explicit precedence. Complete 02-Library first.

Understand the idea

Configuration changes behavior without editing business logic. The project copies settings.json into the build output directory, where the application can locate it relative to AppContext.BaseDirectory. This lesson loads a JSON default and then applies one environment-variable override. It implements that precedence explicitly; a later ASP.NET Core application uses the framework configuration system instead. Environment variables are text, so parse and validate them. Keep credentials out of repository configuration files. A malformed JSON file or invalid override stops this program with a clear exception instead of silently changing the target. Relative paths from the terminal are less reliable than application-relative paths.

Step by step

1. Open Program.cs and StudySettings.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\03-Configuration
```

Expected result

```
Course: .NET 10; daily target: 30 minutes
```

Exercise

Run with a daily target of 45 using an environment variable. Restore its previous value afterward.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Environment changes here affect the current terminal and its child processes.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-03-environment-override.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-03-environment-override
# Implement the exercise, then verify:
dotnet run --project .\DotNet10\03-Configuration
git diff -- DotNet10/03-Configuration/README.md DotNet10/03-Configuration/Program.cs
git add -- DotNet10/03-Configuration/README.md DotNet10/03-Configuration/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(dotnet10): verify the daily target override"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-03-environment-override`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.03.Configuration.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup><None Update="settings.json" CopyToOutputDirectory="PreserveNewest" /></ItemGroup>
</Project>
```

Program.cs

```
using System.Text.Json;
string path = Path.Combine(AppContext.BaseDirectory, "settings.json");
var settings = JsonSerializer.Deserialize<StudySettings>(File.ReadAllText(path))
    ?? throw new InvalidDataException("Settings cannot be null.");
string? overrideValue = Environment.GetEnvironmentVariable("TRAINING_DAILY_MINUTES");
int dailyMinutes = settings.DailyMinutes;
if (overrideValue is not null && !int.TryParse(overrideValue, out dailyMinutes)) {
  throw new InvalidDataException("TRAINING_DAILY_MINUTES must be a whole number.");
}

if (dailyMinutes is < 1 or > 180) {
  throw new InvalidDataException("DailyMinutes must be between 1 and 180.");
}

Console.WriteLine($"Course: {settings.Course}; daily target: {dailyMinutes} minutes");
```

settings.json

```
{
  "Course": ".NET 10",
  "DailyMinutes": 30
}
```

StudySettings.cs

```
public sealed record StudySettings(string Course, int DailyMinutes);
```

03-Configuration exercise solution

Run with a daily target of 45 using an environment variable. Restore its previous value afterward.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$previousTarget = $env:TRAINING_DAILY_MINUTES
try {
  $env:TRAINING_DAILY_MINUTES = '45'
  dotnet run --project .\DotNet10\03-Configuration
} finally {
  $env:TRAINING_DAILY_MINUTES = $previousTarget
}
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

04-DependencyInjection: Dependency injection and logging

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn containers, service lifetimes, disposal, and structured logs. Complete 03-Configuration first.

Understand the idea

A dependency injection container stores registrations and constructs objects with their dependencies. AddSingleton registers one instance for the provider lifetime. AddTransient creates a new instance when resolved. AddScoped creates one instance per scope, commonly an HTTP request. Do not inject a scoped service directly into a singleton; their lifetimes conflict. GetRequiredService fails clearly if a service is not registered. Disposing the provider disposes services it owns. ILogger uses message templates: {Target} names a structured value. This console app references Microsoft.AspNetCore.App to use the installed shared-framework assemblies without NuGet downloads. Logging is diagnostic output; business methods still return data.

Step by step

1. Open Program.cs and IGoalService.cs, GoalService.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\04-DependencyInjection
```

Expected result

```
Remaining: 40 minutes, plus an information log about the target. Log ordering can vary.
```

Exercise

Resolve IGoalService twice and check ReferenceEquals. Change its registration to AddTransient and repeat.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

The container lifetime controls instance identity.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-04-service-lifetimes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-04-service-lifetimes
# Implement the exercise, then verify:
dotnet run --project .\DotNet10\04-DependencyInjection
git diff -- DotNet10/04-DependencyInjection/README.md DotNet10/04-DependencyInjection/Program.cs
git add -- DotNet10/04-DependencyInjection/README.md DotNet10/04-DependencyInjection/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(dotnet10): compare service lifetimes"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-04-service-lifetimes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.04.DependencyInjection.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup><FrameworkReference Include="Microsoft.AspNetCore.App" /></ItemGroup>
</Project>
```

GoalService.cs

```
using Microsoft.Extensions.Logging;

public sealed class GoalService : IGoalService {
  private readonly ILogger<GoalService> _logger;

  public GoalService(ILogger<GoalService> logger) {
    _logger = logger;
  }

  public int Remaining(int completed, int target) {
    if (completed < 0 || target < 0) {
      throw new ArgumentOutOfRangeException(nameof(completed));
    }
    _logger.LogInformation("Calculating remaining minutes for target {Target}", target);
    return Math.Max(0, target - completed);
  }
}
```

IGoalService.cs

```
public interface IGoalService {
  int Remaining(int completed, int target);
}
```

Program.cs

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

var services = new ServiceCollection();
services.AddLogging(builder => builder.AddSimpleConsole(options => options.SingleLine = true));
services.AddSingleton<IGoalService, GoalService>();
using ServiceProvider provider = services.BuildServiceProvider();
var goal = provider.GetRequiredService<IGoalService>();
Console.WriteLine($"Remaining: {goal.Remaining(20, 60)} minutes");
```

04-DependencyInjection exercise solution

Resolve IGoalService twice and check ReferenceEquals. Change its registration to AddTransient and repeat.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
var first = provider.GetRequiredService<IGoalService>();
var second = provider.GetRequiredService<IGoalService>();
Console.WriteLine(ReferenceEquals(first, second));
// Singleton: True. After changing to AddTransient: False.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

05-MinimalApi: Your first HTTP API

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn http routes, responses, json, and a server process. Complete 04-DependencyInjection first.

Understand the idea

An API lets another program request behavior through a contract. ASP.NET Core hosts the HTTP server. A route matches a path and HTTP method. MapGet registers a handler for GET. A path parameter such as {name} is bound to a handler parameter. Results.Ok returns status 200 and serializes an object as JSON. The server keeps running until you press Ctrl+C. Start it in one terminal and send requests from another. The run command binds only to the local machine using a loopback address. A port can be used by only one listening process; choose a different port if it is already occupied.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. If the example is a server, leave it running and use a second terminal for requests; stop it with Ctrl+C afterward.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\05-MinimalApi -- --urls http://127.0.0.1:5081
# In a second terminal:
Invoke-RestMethod http://127.0.0.1:5081/health
Invoke-RestMethod http://127.0.0.1:5081/hello/Dejan
```

Expected result

```
Health returns status=ok. Hello returns message=Hello, Dejan! The server prints startup logs.
```

Exercise

Add GET /minutes/{count:int} returning hours. Return 400 for a negative count.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits. Restart the server after editing its code.

Hint

An int route constraint limits which paths match; a value rule still needs validation.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-05-minutes-route.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-05-minutes-route
# Implement the exercise, then verify:
# Run this lesson's README server/client checks in the required terminals.
git diff -- DotNet10/05-MinimalApi/README.md DotNet10/05-MinimalApi/Program.cs
git add -- DotNet10/05-MinimalApi/README.md DotNet10/05-MinimalApi/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(dotnet10): add a minutes conversion endpoint"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-05-minutes-route`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.05.MinimalApi.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
// Keep local learning logs in the terminal rather than the Windows Event Log.
builder.Logging.ClearProviders();
builder.Logging.AddConsole();
var app = builder.Build();
app.MapGet("/health", () => Results.Ok(new { Status = "ok" }));
app.MapGet("/hello/{name}", (string name) => Results.Ok(new { Message = $"Hello, {name}!" }));
app.Run();
```

05-MinimalApi exercise solution

Add GET /minutes/{count:int} returning hours. Return 400 for a negative count.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
app.MapGet("/minutes/{count:int}", (int count) =>
    count < 0
        ? Results.BadRequest(new { Error = "Count cannot be negative." })
        : Results.Ok(new { Hours = count / 60.0 }));
// Insert before app.Run().
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

06-Validation: Validate request bodies and status codes

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn body binding, validation responses, and api design. Complete 05-MinimalApi first.

Understand the idea

ASP.NET Core binds a complex request parameter from a JSON body. A nullable property honestly models missing or null input. Binding checks representation, such as whether a number is valid JSON, while domain validation checks meaning. This example explicitly validates fields inside the handler rather than depending on implicit validation conventions. ValidationProblem returns a structured 400 response. The valid request returns 200 because this endpoint only validates and echoes data; it does not persist a resource. The capstone uses 201 for a created task and 404 for a missing ID. Clients should inspect status codes and handle failures.

Step by step

1. Open Program.cs and CreateSession.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. If the example is a server, leave it running and use a second terminal for requests; stop it with Ctrl+C afterward.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\06-Validation -- --urls http://127.0.0.1:5082
# Second terminal:
$body = @{
  topic = 'CSharp'
  minutes = 30
} | ConvertTo-Json
Invoke-RestMethod http://127.0.0.1:5082/sessions -Method Post -ContentType application/json -Body $body
```

Expected result

```
A valid request returns topic=CSharp and minutes=30. Blank topics or minutes=0 return HTTP 400.
```

Exercise

Make two requests: one valid and one with minutes=0. Catch the expected client exception for the invalid request.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits. Restart the server after editing its code.

Hint

Invoke-RestMethod throws for a failing HTTP status.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-06-invalid-input.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-06-invalid-input
# Implement the exercise, then verify:
# Run this lesson's README server/client checks in the required terminals.
git diff -- DotNet10/06-Validation/README.md DotNet10/06-Validation/Program.cs
git add -- DotNet10/06-Validation/README.md DotNet10/06-Validation/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(dotnet10): verify invalid session requests"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-06-invalid-input`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?

- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CreateSession.cs

```
public sealed record CreateSession(string? Topic, int Minutes);
```

DotNet10.06.Validation.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
// Keep local learning logs in the terminal rather than the Windows Event Log.
builder.Logging.ClearProviders();
builder.Logging.AddConsole();
var app = builder.Build();
app.MapPost("/sessions", ValidateSession);
app.Run();

static IResult ValidateSession(CreateSession request) {
  if (string.IsNullOrWhiteSpace(request.Topic) || request.Topic.Length > 100) {
    return Results.ValidationProblem(new Dictionary<string, string[]> {
      ["topic"] = ["Use 1 to 100 nonblank characters."]
    });
  }

  if (request.Minutes is < 1 or > 180) {
    return Results.ValidationProblem(new Dictionary<string, string[]> {
      ["minutes"] = ["Use a value from 1 to 180."]
    });
```

```
  }

  return Results.Ok(new {
    Topic = request.Topic.Trim(),
    Minutes = request.Minutes
  });
}
```

06-Validation exercise solution

Make two requests: one valid and one with minutes=0. Catch the expected client exception for the invalid request.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
try {
  $body = @{
    topic = 'CSharp'
    minutes = 0
  } | ConvertTo-Json
  $request = @{
    Uri = 'http://127.0.0.1:5082/sessions'
    Method = 'Post'
    ContentType = 'application/json'
    Body = $body
  }
  Invoke-RestMethod @request
} catch {
  Write-Output "Expected validation failure: $($_.Exception.Message)"
}
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

07-HttpClient: Call an API from C#

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn http clients, timeouts, deserialization, and failure handling. Complete 06-Validation first.

Understand the idea

HttpClient sends requests. GetFromJsonAsync combines a GET request, successful-status checking, and JSON deserialization. A BaseAddress provides the server origin; the relative path selects an endpoint. A timeout limits how long a request can wait, and a cancellation token lets a caller stop it. The response type describes the JSON fields you expect. A successful HTTP response can still contain null or incompatible data, so consider both transport and content errors. This short console application owns one HttpClient for its lifetime. Long-running services should reuse clients or use IHttpClientFactory rather than construct and dispose one per request.

Step by step

1. Open Program.cs and HealthResponse.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. If the example is a server, leave it running and use a second terminal for requests; stop it with Ctrl+C afterward.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
# Start lesson 05 in a separate terminal first.
dotnet run --project .\DotNet10\07-HttpClient -- http://127.0.0.1:5081
```

Expected result

```
Server status: ok. If the server is stopped, the client reports an error and exits with code 1.
```

Exercise

Stop the server, run the client, and inspect LASTEXITCODE. Restart the server and verify success.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Check the process exit code immediately after dotnet.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-07-connection-failure.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-07-connection-failure
# Implement the exercise, then verify:
# Run this lesson's README server/client checks in the required terminals.
git diff -- DotNet10/07-HttpClient/README.md DotNet10/07-HttpClient/Program.cs
git add -- DotNet10/07-HttpClient/README.md DotNet10/07-HttpClient/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(dotnet10): document client failure behavior"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-07-connection-failure`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.07.HttpClient.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

HealthResponse.cs

```
public sealed record HealthResponse(string Status);
```

Program.cs

```
using System.Net.Http.Json;
using System.Text.Json;

string baseUrl = args.Length > 0 ? args[0] : "http://127.0.0.1:5081";
using var http = new HttpClient {
  BaseAddress = new Uri(baseUrl),
  Timeout = TimeSpan.FromSeconds(5)
};
using var cancellation = new CancellationTokenSource(TimeSpan.FromSeconds(5));
try {
  var response = await http.GetFromJsonAsync<HealthResponse>("/health", cancellation.Token)
      ?? throw new InvalidDataException("The server returned null.");
  Console.WriteLine($"Server status: {response.Status}");
} catch (Exception error) when (
    error is HttpRequestException or OperationCanceledException or JsonException
        or InvalidDataException) {
  Console.Error.WriteLine($"Request failed: {error.Message}");
  Environment.ExitCode = 1;
}
```

07-HttpClient exercise solution

Stop the server, run the client, and inspect LASTEXITCODE. Restart the server and verify success.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
dotnet run --project .\DotNet10\07-HttpClient -- http://127.0.0.1:5081
Write-Output "Exit code: $LASTEXITCODE"
## First try with the server stopped, then with it running.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

08-Testing: Executable checks and failure cases

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn assertions, boundaries, persistence checks, and test runners. Complete 07-HttpClient first.

Understand the idea

A useful test exercises a contract rather than repeating its implementation. Equal compares actual and expected values. Throws verifies rejected inputs. Check catches failures to let the rest of the suite run, and the program returns a nonzero code when any check fails. Boundary cases include zero, negative values, limits, and overflow. Persistence checks construct a new repository after writing so an in-memory result cannot conceal a storage failure. The test uses a uniquely named temporary directory and cleans up only that directory. This is a package-free executable test harness, run with dotnet run rather than dotnet test. Professional teams often use xUnit, NUnit, or MSTest and add richer tooling.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\08-Testing
```

Expected result

```
Each contract prints PASS, followed by Failures: 0. A failing check prints FAIL and returns exit code
1.
```

Exercise

Introduce a deliberate bug in StudyBudget, run the tests, confirm failure, then restore the correct rule.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Remove the negative-input guard temporarily and check which test catches it.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-08-boundary-cases.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-08-boundary-cases
# Implement the exercise, then verify:
dotnet run --project .\DotNet10\08-Testing
git diff -- DotNet10/08-Testing/README.md DotNet10/08-Testing/Program.cs
git add -- DotNet10/08-Testing/README.md DotNet10/08-Testing/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(dotnet10): check study budget boundaries"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-08-boundary-cases`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

DotNet10.08.Testing.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../02-Library/Library/StudyMath.csproj" />
    <ProjectReference Include="../10-Capstone/DotNet10.10.Capstone.csproj" />
  </ItemGroup>
</Project>
```

Program.cs

```
using LearningTasks;
using StudyMath;

int failures = 0;
Check("budget multiplies", () => Equal(75, StudyBudget.TotalMinutes(3, 25)));
Check("goal boundary", () => Equal(true, StudyBudget.HasReachedGoal(60, 60)));
Check("negative sessions rejected",
    () => Throws<ArgumentOutOfRangeException>(() => StudyBudget.TotalMinutes(-1, 25)));
Check("overflow rejected",
    () => Throws<OverflowException>(() => StudyBudget.TotalMinutes(int.MaxValue, 2)));
Check("blank task rejected", () => Equal(false, TaskRules.IsValidTitle("   ")));
Check("long task rejected", () => Equal(false, TaskRules.IsValidTitle(new string('a', 201))));
Check("valid task accepted", () => Equal(true, TaskRules.IsValidTitle("Learn C#")));

string directory = Path.Combine(Path.GetTempPath(), "LearningPack-" + Guid.NewGuid().ToString("N"));
Directory.CreateDirectory(directory);
try {
  string path = Path.Combine(directory, "tasks.json");
  var repository = new TaskRepository(path);
  var created = repository.Add("Learn C#");
  Check("task starts incomplete", () => Equal(false, created.IsComplete));
  Check("completion persists", () => {
    repository.Complete(created.Id);
    var reloaded = new TaskRepository(path);
    Equal(true, reloaded.GetAll().Single().IsComplete);
  });
  Check("missing id returns null", () => Equal<StudyTask?>(null, repository.Complete(999)));
```

```
  Check("deletion persists", () => {
    Equal(true, repository.Delete(created.Id));
    Equal(0, new TaskRepository(path).GetAll().Count);
  });
  Check("IDs are never reused", () => Equal(2, repository.Add("Second task").Id));
  Check("corrupt storage fails visibly", () => {
    File.WriteAllText(path, "{ invalid json");
    Throws<System.Text.Json.JsonException>(() => { _ = new TaskRepository(path); });
  });
} finally {
  // This process created this unique directory and deletes only its own test data.
  Directory.Delete(directory, recursive: true);
}
Console.WriteLine($"Failures: {failures}");
return failures == 0 ? 0 : 1;

void Check(string name, Action action) {
  try {
    action();
    Console.WriteLine($"PASS: {name}");
  } catch (Exception error) {
    failures++;
    Console.Error.WriteLine($"FAIL: {name}: {error.Message}");
  }
}
static void Equal<T>(T expected, T actual) {
  if (!EqualityComparer<T>.Default.Equals(expected, actual)) {
    throw new InvalidOperationException($"Expected {expected}, got {actual}.");
  }
}
static void Throws<T>(Action action) where T : Exception {
  try {
    action();
  } catch (T) {
    return;
  }
  throw new InvalidOperationException($"Expected {typeof(T).Name}.");
}
```

08-Testing exercise solution

Introduce a deliberate bug in StudyBudget, run the tests, confirm failure, then restore the correct rule.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
## From the repository root:
```

```
dotnet run --project .\DotNet10\08-Testing
$testExit = $LASTEXITCODE
Write-Output "Test exit: $testExit"
## Restore the library before moving on. A successful run exits 0.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

09-Publishing: Release builds and framework-dependent publishing

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn build configurations, deployment output, and runtime requirements. Complete 08-Testing first.

Understand the idea

A Debug build favors development. A Release build enables optimizations intended for deployment. dotnet publish prepares application files for distribution. Framework-dependent publishing expects the target machine to have a compatible runtime installed. Running the DLL through dotnet makes that dependency explicit. A self-contained publish includes a runtime for a selected runtime identifier, such as win-x64, and usually requires downloading runtime packs. Native AOT is another deployment option with restrictions and toolchain requirements. This course publishes the small lesson 01 console app without a runtime download. Publishing locally creates files; it does not deploy to a server or expose an application on the internet.

Step by step

1. Open Publish.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\DotNet10\09-Publishing\Publish.ps1
```

Expected result

```
Release/publish build output, followed by .NET major version: 10 and the SDK/runtime explanation.
```

Exercise

Compare bin/Debug and the published output. Locate the DLL, runtimeconfig.json, and deps.json and explain each role.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

runtimeconfig names the framework; deps describes dependencies.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-09-publish-notes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-09-publish-notes
# Implement the exercise, then verify:
.\DotNet10\09-Publishing\Publish.ps1
git diff -- DotNet10/09-Publishing/README.md DotNet10/09-Publishing/Publish.ps1
git add -- DotNet10/09-Publishing/README.md DotNet10/09-Publishing/Publish.ps1
git diff --cached
git diff --cached --check
git commit -m "docs(dotnet10): explain published runtime files"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-09-publish-notes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Publish.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$project = Join-Path $PSScriptRoot '..\01-Sdk\DotNet10.01.Sdk.csproj'
$destination = Join-Path $PSScriptRoot 'output\published'
& dotnet publish $project -c Release --self-contained false -o $destination -m:1 -nr:false
if ($LASTEXITCODE -ne 0) {
  throw "Publish failed: $LASTEXITCODE"
}
& dotnet (Join-Path $destination 'DotNet10.01.Sdk.dll')
if ($LASTEXITCODE -ne 0) {
  throw "Published app failed: $LASTEXITCODE"
}
```

09-Publishing exercise solution

Compare bin/Debug and the published output. Locate the DLL, runtimeconfig.json, and deps.json and explain each role.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
Get-ChildItem .\DotNet10\09-Publishing\output\published
Get-Content .\DotNet10\09-Publishing\output\published\DotNet10.01.Sdk.runtimeconfig.json
Get-Content .\DotNet10\09-Publishing\output\published\DotNet10.01.Sdk.deps.json
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

10-Capstone: A persistent task API

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn routes, validated contracts, persistent state, and concurrency. Complete 09-Publishing first.

Understand the idea

This project combines a minimal API, dependency injection, domain rules, immutable records, and a JSON repository. Endpoints coordinate requests; TaskRules defines title validity; TaskRepository owns storage. It uses a lock to serialize changes within one server process, writes a candidate state to a temporary sibling file, replaces the saved file, and only then updates in-memory state. IDs use a persisted counter and are not reused after deletion. Invalid storage stops startup visibly. Returned records and arrays prevent callers from mutating repository internals. This is a local, single-process teaching server. Multiple server processes sharing its file need stronger storage coordination; real public services also need authentication, a database, monitoring, and deployment configuration.

Step by step

1. Open Program.cs and TaskRepository.cs, TaskRules.cs, StudyTask.cs, StoredTasks.cs, CreateTask.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. If the example is a server, leave it running and use a second terminal for requests; stop it with Ctrl+C afterward.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\DotNet10\10-Capstone -- --urls http://127.0.0.1:5080
# In a second terminal:
.\PowerShell\10-Capstone\Demo.ps1 -Action Add -Title 'Learn .NET 10'
.\PowerShell\10-Capstone\Demo.ps1 -Action List
```

Expected result

```
POST creates a task with HTTP 201; list returns an array. Completion returns 200, deletion 204, missing
```

```
IDs 404, and invalid titles 400. Restarting with the same data path keeps tasks.
```

Exercise

Add an endpoint that returns total and completed task counts. Extend the root API smoke checks to verify it.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits. Restart the server after editing its code.

Hint

Use a repository snapshot and count matching records.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/dotnet10-10-task-summary.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/dotnet10-10-task-summary
# Implement the exercise, then verify:
# Run this lesson's README server/client checks in the required terminals.
git diff -- DotNet10/10-Capstone/README.md DotNet10/10-Capstone/Program.cs
git add -- DotNet10/10-Capstone/README.md DotNet10/10-Capstone/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(dotnet10): add task summary counts"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/dotnet10-10-task-summary`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CreateTask.cs

```
public sealed record CreateTask(string? Title);
```

DotNet10.10.Capstone.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
using LearningTasks;

var builder = WebApplication.CreateBuilder(args);
// Keep local learning logs in the terminal rather than the Windows Event Log.
builder.Logging.ClearProviders();
builder.Logging.AddConsole();
string dataPath = builder.Configuration["TrainingDataPath"]
    ?? Path.Combine(AppContext.BaseDirectory, "output", "tasks.json");
builder.Services.AddSingleton(new TaskRepository(dataPath));
var app = builder.Build();

app.MapGet("/health", () => Results.Ok(new { Status = "ok" }));
app.MapGet("/tasks", (TaskRepository repository) => Results.Ok(repository.GetAll()));
app.MapGet("/tasks/{id:int}", (int id, TaskRepository repository) =>
    repository.GetAll().FirstOrDefault(task => task.Id == id) is { } task
        ? Results.Ok(task) : Results.NotFound());
app.MapPost("/tasks", CreateStudyTask);
app.MapPut("/tasks/{id:int}/complete", (int id, TaskRepository repository) =>
    repository.Complete(id) is { } task ? Results.Ok(task) : Results.NotFound());
app.MapDelete("/tasks/{id:int}", (int id, TaskRepository repository) =>
    repository.Delete(id) ? Results.NoContent() : Results.NotFound());
app.Run();

static IResult CreateStudyTask(CreateTask request, TaskRepository repository) {
```

```
  if (!TaskRules.IsValidTitle(request.Title)) {
    return Results.ValidationProblem(new Dictionary<string, string[]> {
      ["title"] = ["Use 1 to 200 nonblank characters."]
    });
  }

  var task = repository.Add(request.Title!);
  return Results.Created($"/tasks/{task.Id}", task);
}
```

StoredTasks.cs

```
namespace LearningTasks;

public sealed record StoredTasks(int NextId, List<StudyTask> Tasks);
```

StudyTask.cs

```
namespace LearningTasks;

public sealed record StudyTask(int Id, string Title, bool IsComplete);
```

TaskRepository.cs

```
using System.Text.Json;

namespace LearningTasks;

public sealed class TaskRepository {
  private static readonly JsonSerializerOptions _jsonOptions = new() { WriteIndented = true };
  private readonly object _gate = new();
  private readonly string _path;
  private List<StudyTask> _tasks;
  private int _nextId;

  public TaskRepository(string path) {
    _path = Path.GetFullPath(path);
    if (File.Exists(_path)) {
      var stored = JsonSerializer.Deserialize<StoredTasks>(File.ReadAllText(_path))
          ?? throw new InvalidDataException("Task storage cannot be null.");
      if (stored.Tasks is null || stored.NextId < 1
          || stored.Tasks.Any(task => task is null || task.Id < 1
              || !TaskRules.IsValidTitle(task.Title))
```

```
        || stored.Tasks.Select(task => task.Id).Distinct().Count() != stored.Tasks.Count
        || stored.Tasks.Any(task => task.Id >= stored.NextId)) {
      throw new InvalidDataException("Task storage is invalid.");
    }
    _tasks = stored.Tasks;
    _nextId = stored.NextId;
  } else {
    _tasks = [];
    _nextId = 1;
  }
}

public IReadOnlyList<StudyTask> GetAll() {
  lock (_gate) {
    return _tasks.ToArray();
  }
}

public StudyTask Add(string title) {
  if (!TaskRules.IsValidTitle(title)) {
    throw new ArgumentException("Invalid title.", nameof(title));
  }
  lock (_gate) {
    int followingId = checked(_nextId + 1);
    var task = new StudyTask(_nextId, title.Trim(), false);
    List<StudyTask> candidate = [.. _tasks, task];
    Save(candidate, followingId);
    _tasks = candidate;
    _nextId = followingId;
    return task;
  }
}

public StudyTask? Complete(int id) {
  lock (_gate) {
    int index = _tasks.FindIndex(task => task.Id == id);
    if (index < 0) {
      return null;
    }
    var updated = _tasks[index] with {
      IsComplete = true
    };
    var candidate = _tasks.ToList();
    candidate[index] = updated;
    Save(candidate, _nextId);
    _tasks = candidate;
    return updated;
  }
}

public bool Delete(int id) {
  lock (_gate) {
```

```
      var candidate = _tasks.Where(task => task.Id != id).ToList();
      if (candidate.Count == _tasks.Count) {
        return false;
      }
      Save(candidate, _nextId);
      _tasks = candidate;
      return true;
    }
  }

  private void Save(List<StudyTask> candidate, int followingId) {
    Directory.CreateDirectory(Path.GetDirectoryName(_path)!);
    string temp = _path + ".tmp";
    var snapshot = new StoredTasks(followingId, candidate);
    File.WriteAllText(temp, JsonSerializer.Serialize(snapshot, _jsonOptions));
    File.Move(temp, _path, overwrite: true);
  }
}
```

TaskRules.cs

```
namespace LearningTasks;

public static class TaskRules {
  private const int _maxTitleLength = 200;

  public static bool IsValidTitle(string? title) {
    return !string.IsNullOrWhiteSpace(title) && title.Length <= _maxTitleLength;
  }
}
```

10-Capstone exercise solution

Add an endpoint that returns total and completed task counts. Extend the root API smoke checks to verify it.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
app.MapGet("/tasks/summary", (TaskRepository repository) => {
  var tasks = repository.GetAll();
  return Results.Ok(new { Total = tasks.Count, Completed = tasks.Count(t => t.IsComplete) });
});
// Insert before app.Run(). The literal summary route does not match {id:int}.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.